SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Modelling Comparative Analysis Assignment

Code of Conduct:

I G.Karthik Reddy-192525047 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G.Karthik Reddy**

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	**Automobile Fault Diagnosis:** An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	**Industrial Machine Fault Diagnosis:** A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	**Cybersecurity Threat Detection:** A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	**Healthcare Diagnosis:** A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	**Agricultural Crop Disease Diagnosis:** Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	**Banking Loan Eligibility:** A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities

	of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

1. Title

Cybersecurity Threat Detection Using Rule-Based Expert System

2. Problem Statement

A Security Operations Center (SOC) wants to automatically identify possible cybersecurity threats from security events such as:

- Repeated login failures
- Unusual login locations
- Privilege escalation
- Suspicious file access
- Abnormal network traffic

The system should represent this knowledge using **production rules, propositional logic, first-order logic, and Prolog**, and compare **forward and backward reasoning** for deriving security conclusions.

3. Objectives

The objectives of this modelling exercise are:

1. Identify cybersecurity entities and security events.
2. Represent security knowledge using facts and rules.
3. Develop IF–THEN production rules.
4. Model the problem using propositional logic.
5. Model the problem using first-order logic.
6. Implement the knowledge base in Prolog.
7. Demonstrate forward chaining.
8. Demonstrate backward chaining.
9. Compare the four modelling approaches.
10. Select the most suitable approach for cybersecurity threat detection.

These areas directly match the report structure specified in the document.

4. Domain Knowledge and Facts

The major security events are:

Security Event	Description
Repeated Login Failures	Multiple unsuccessful login attempts
Unusual Location	Login from an unexpected location
Privilege Escalation	User obtains higher privileges
Suspicious File Access	Access to sensitive files unexpectedly
Abnormal Network Traffic	Unusual network communication
Security Threat	Possible malicious activity

Example Scenario

Suppose a user account has:

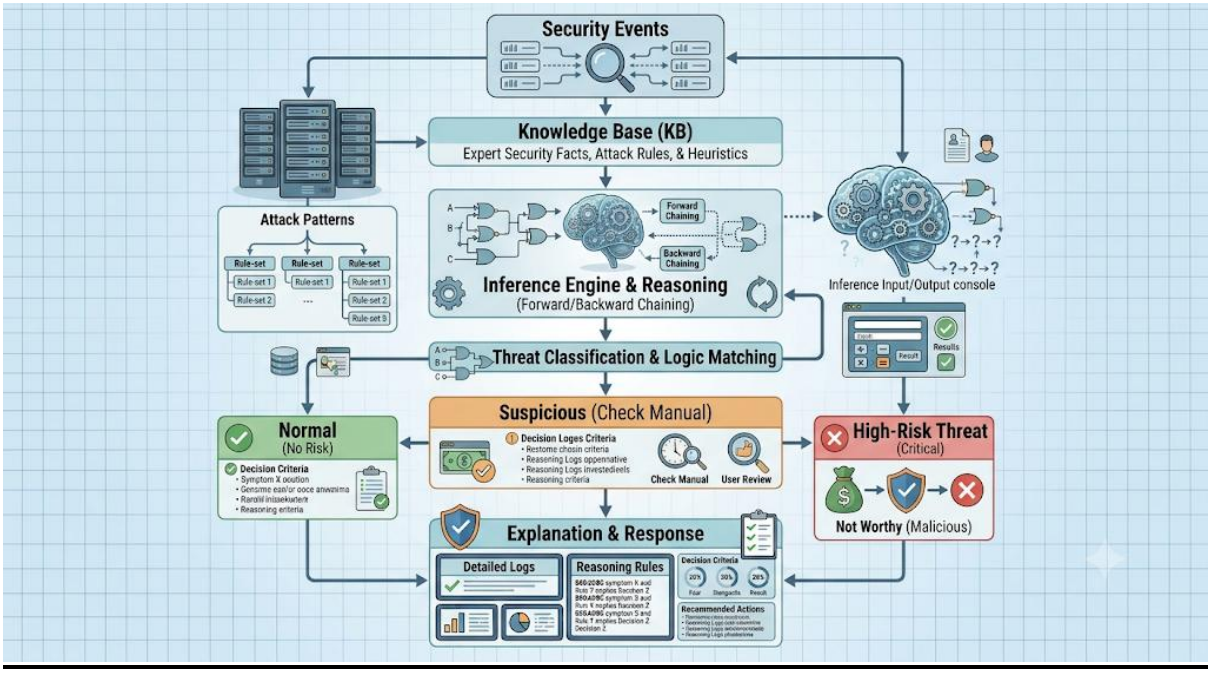
5 failed login attempts

+

Login from an unusual location

+
Suspicious file access
The system can infer:
Potential Security Threat

5. Suitable System Architecture



6. Production Rule Model

Production rules represent knowledge in **IF–THEN** form.

Rule 1 – Repeated Login Failure

IF

a user has repeated login failures

THEN

mark the user as suspicious.

Rule 2 – Unusual Location

IF

a user logs in from an unusual location

THEN

mark the login as suspicious.

Rule 3 – Privilege Escalation

IF

a normal user suddenly obtains administrative privileges

THEN

generate a security alert.

Rule 4 – Suspicious File Access

IF
a user accesses sensitive files unexpectedly

THEN
generate a security alert.

Rule 5 – Combined Threat

IF
repeated login failures
AND unusual login location
AND suspicious file access

THEN
identify a potential security threat.

7. Propositional Logic Model

In propositional logic, each statement is represented using a proposition.
Let:

- L= Repeated login failures
- U= Unusual login location
- P= Privilege escalation
- F= Suspicious file access
- N= Abnormal network traffic
- T= Security threat

Rule 1

If repeated login failures and unusual location occur:

$$L \wedge U \rightarrow T$$

Rule 2

If privilege escalation and suspicious file access occur:

$$P \wedge F \rightarrow T$$

Rule 3

If abnormal network traffic occurs:

$$N \rightarrow T$$

Example

Given:

$$L=True$$
$$U=True$$

Then:

$$L \wedge U=True$$

Therefore:

$$T=True$$

Conclusion

The system identifies a **potential security threat**.

8. First-Order Logic Model

First-Order Logic (FOL) is more expressive because it can represent **entities, relationships, properties, variables, and rules**. The assignment specifically requires these elements in the FOL model.

Predicates

failed_login(User)
unusual_location(User)
privilege_escalation(User)
suspicious_file_access(User)
abnormal_traffic(User)
security_threat(User)

Rule 1

For every user:
$\forall x[(failed\_login(x) \wedge unusual\_location(x)) \rightarrow security\_threat(x)]$
Meaning:
If any user has repeated login failures and an unusual login location, that user may represent a security threat.
Rule 2
$\forall x[(privilege\_escalation(x) \wedge suspicious\_file\_access(x)) \rightarrow security\_threat(x)]$
Rule 3
$\forall x[abnormal\_traffic(x) \rightarrow security\_threat(x)]$

9. Example FOL Facts

For user **karthik**:

failed_login(karthik)
unusual_location(karthik)

Therefore:

security_threat(karthik)

because:
$failed\_login(karthik) \wedge unusual\_location(karthik) \rightarrow security\_threat(karthik)$

10. Prolog Model

The same knowledge can be represented using Prolog facts and rules.

Complete SWI-Prolog Program

```
/* ======================================
   CYBERSECURITY THREAT DETECTION SYSTEM
   ====================================== */

/* ---------- FACTS ---------- */

/* User Karthik */
failed_login(karthik).
unusual_location(karthik).
```

```
suspicious_file_access(karthik).

/* User Ravi */
privilege_escalation(ravi).
suspicious_file_access(ravi).

/* User Anu */
abnormal_traffic(anu).

/* ---------- RULES ---------- */

/* Rule 1 */
security_threat(User) :-
   failed_login(User),
   unusual_location(User).

/* Rule 2 */
security_threat(User) :-
   privilege_escalation(User),
   suspicious_file_access(User).

/* Rule 3 */
security_threat(User) :-
   abnormal_traffic(User).

/* ---------- THREAT LEVEL ---------- */

threat_level(User, high) :-
   security_threat(User),
   suspicious_file_access(User).

threat_level(User, medium) :-
   security_threat(User).
```

11. Prolog Queries

Query 1

```
?- security_threat(karthik).
```

Output

```
true.
Because:
failed_login(karthik)
+
unusual_location(karthik)
satisfies:
```

```
security_threat(User) :-
   failed_login(User),
   unusual_location(User).
```

Query 2

?- security_threat(ravi).

Output

true.

Because:

privilege_escalation(ravi)

+

suspicious_file_access(ravi)

indicates a potential threat.

Query 3

?- security_threat(anu).

Output

true.

Because:

abnormal_traffic(anu)

matches the third rule.

12. Forward Chaining

Forward chaining starts with **known facts** and moves toward a conclusion.

Example: Karthik

Initial facts:

failed_login(karthik)

unusual_location(karthik)

suspicious_file_access(karthik)

Step 1

Check:

failed_login(karthik)

→ TRUE

Step 2

Check:

unusual_location(karthik)

→ TRUE

Step 3

Apply Rule 1:

IF failed login

AND unusual location

THEN security threat

Therefore:

security_threat(karthik)

Step 4

Since suspicious file access is also present:

threat_level(karthik, high)

Inference Sequence

Failed Login

+

Unusual Location

↓

Security Threat

+

Suspicious File Access

↓

High Threat Level

13. Backward Chaining

Backward chaining starts with a **goal** and works backward to find supporting facts.

Goal

?- security_threat(karthik).

Prolog searches for a rule that can prove:

security_threat(karthik)

It finds:

```
security_threat(User) :-
   failed_login(User),
   unusual_location(User).
```

Now Prolog checks:

failed_login(karthik) → TRUE

unusual_location(karthik) → TRUE

Therefore:

security_threat(karthik)

is proved.

Backward Reasoning

Goal:

Security Threat?

↓

Need:

Failed Login + Unusual Location

↓

Check Facts

↓

Both TRUE

↓

Security Threat = TRUE

14. Unification

Prolog uses unification to match variables with values.
Consider:
security_threat(User).
When the query is:
?- security_threat(karthik).
Prolog unifies:
and checks the corresponding facts.
Thus:
security_threat(User)
becomes:
security_threat(karthik)

15. Backtracking

Suppose we ask:
?- security_threat(User).
Prolog searches for all possible users.
Possible results:
User = karthik ;
User = ravi ;
User = anu.
Prolog uses **backtracking** to return alternative solutions.

16. Comparative Analysis

The assignment requires comparison based on knowledge representation, expressiveness, rule representation, inference mechanism, forward/backward chaining, relationships, scalability, explainability, implementation ease, and suitability.

Parameter	**Production Rules**	**Propositional Logic**	**First-Order Logic**	**Prolog**
Knowledge Representation	IF–THEN rules	Propositions	Predicates, variables and quantifiers	Facts + rules
Expressiveness	Moderate	Low–Moderate	**High**	**High**
Rule Representation	Very simple	Logical implications	Predicate-based rules	Natural rule syntax
Inference Mechanism	Rule matching	Logical inference	Predicate inference	Unification + resolution
Forward Chaining	Excellent	Possible	Possible	Can be implemented
Backward Chaining	Possible	Limited	Possible	**Excellent / Built-in style**
Relationships	Limited	Limited	**Strong**	**Strong**
Scalability	Moderate	Limited for complex domains	Can become complex	Good for rule-based systems
Explainability	**Very High**	High	High	**High**

Ease of Implementation	Very Easy	Easy	Moderate	Easy–Moderate
Flexibility	Moderate	Low	**Very High**	**Very High**
Cybersecurity Suitability	Good	Moderate	Very Good	**Very Good**

17. Strengths and Limitations

Production Rules

Strengths

- Simple IF–THEN structure.
- Easy to understand.
- Highly explainable.
- Easy to modify.

Limitations

- Difficult to represent complex relationships.
- Large rule bases can become difficult to maintain.

Propositional Logic

Strengths

- Simple mathematical representation.
- Clear logical reasoning.
- Easy to verify.

Limitations

- Cannot naturally represent individual users and relationships.
- Less expressive for complex cybersecurity environments.

First-Order Logic

Strengths

- Represents individual users.
- Represents relationships.
- Uses variables and quantifiers.
- More expressive.

Limitations

- More complex than propositional logic.
- Reasoning can become computationally expensive for large knowledge bases.

Prolog

Strengths

- Natural representation of facts and rules.
- Supports unification.
- Supports backtracking.
- Excellent for backward reasoning.
- Good for developing expert systems.

Limitations

- Large knowledge bases can become difficult to manage.
- Rule quality directly affects system accuracy.
- Basic rule-based Prolog does not automatically learn new attack patterns from data.

18. Results and Discussion

The model was tested with three users.

User	Observed Events	Result
Karthik	Failed login + unusual location + suspicious file access	High Threat
Ravi	Privilege escalation + suspicious file access	Threat Detected
Anu	Abnormal network traffic	Threat Detected

19. Recommended Model

Prolog is the Recommended Approach

For this cybersecurity expert system, **Prolog provides the best overall implementation approach**.

Reasons

1. It naturally represents facts and rules.
2. It supports logical inference.
3. It provides unification.
4. It supports backtracking.
5. It is well suited to backward reasoning.
6. Rules can be easily modified.
7. It provides understandable and explainable conclusions.

However, in a real Security Operations Center, a rule-based Prolog system would normally be combined with other security technologies and data-driven detection methods rather than used as the only threat-detection mechanism.

20. Limitations and Improvements

Limitations

- The system depends on predefined rules.
- New attack patterns may not be detected unless rules are added.
- Real cybersecurity data can be much more complex.
- False positives and false negatives are possible.
- Large-scale real-time security monitoring requires additional infrastructure.

Future Improvements

The system can be extended with:

- Machine learning.
- Real-time log analysis.
- Network intrusion detection.
- SIEM integration.

- Threat intelligence feeds.
- Automatic rule generation.
- Anomaly detection.
- Web-based security dashboard.

1. Final Conclusion

The **Cybersecurity Threat Detection Expert System** demonstrates four different knowledge-modelling approaches:

Production rules provide a simple IF–THEN representation, propositional logic provides basic logical reasoning, First-Order Logic provides greater expressiveness for entities and relationships, and Prolog provides a practical implementation with **facts, rules, unification, backtracking, and goal-driven reasoning**.

For the selected cybersecurity problem, **Prolog is recommended as the implementation model** because it combines expressive knowledge representation with powerful logical inference mechanisms.

This structure covers all the major sections required by the given document: **problem statement and objectives, domain facts, production rules, propositional logic, First-Order Logic, Prolog implementation, forward/backward chaining, comparative analysis, results, model selection, limitations, and conclusion**.